Отчёт по лабораторной работе №8

Шифр гаммирования

Архипов Александр Сергеевич НБИбд-01-24
1132239106 | 1132239106@rudn.ru

07.10.2026

Содержание

1 Цель работы **1**

2 Теоретические сведения **1**
2.1 Шифр гаммирования 1
2.2 Идея взлома 2

3 Выполнение работы **2**
3.1 Реализация XOR и учебной атаки при повторении ключа 3
3.2 атака на повторно использованный XOR-ключ 3
3.3 точное XOR-шифрование и расшифровка UTF-8 4

4 Выводы **5**

Список литературы **5**

1 Цель работы

Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом.

2 Теоретические сведения

2.1 Шифр гаммирования

Гаммирование – это наложение (снятие) на открытые (зашифрованные) данные криптографической гаммы, т.е. последовательности элементов данных, вырабатываемых с помощью некоторого криптографического алгоритма, для получения зашифрованных (открытых) данных.

Принцип шифрования гаммированием заключается в генерации гаммы шифра с помощью датчика псевдослучайных чисел и наложении полученной гаммы шифра на открытые данные обратимым образом (например, используя операцию сложения по модулю 2). Процесс дешифрования сводится к повторной генерации гаммы шифра при известном ключе и наложении такой же гаммы на зашифрованные данные. Полученный зашифрованный текст является достаточно трудным для раскрытия в том случае, если гамма шифра не содержит повторяющихся битовых последовательностей и изменяется случайным образом для каждого шифруемого слова. Если период гаммы превышает длину всего зашифрованного текста и неизвестна никакая часть исходного текста, то шифр можно раскрыть только прямым перебором (подбором ключа). В этом случае криптостойкость определяется размером ключа.

Метод гаммирования становится бессильным, если известен фрагмент исходного текста и соответствующая ему шифрограмма. В этом случае простым вычитанием по модулю 2 получается отрезок псевдослучайной последовательности и по нему восстанавливается вся эта последовательность.

Метод гаммирования с обратной связью заключается в том, что для получения сегмента гаммы используется контрольная сумма определенного участка шифруемых данных. Например, если рассматривать гамму шифра как объединение непересекающихся множеств H(j), то процесс шифрования можно пердставить следующими шагами:

1. Генерация сегмента гаммы H(1) и наложение его на соответствующий участок шифруемых данных.
2. Подсчет контрольной суммы участка, соответствующего сегменту гаммы H(1).
3. Генерация с учетом контрольной суммы уже зашифрованного участка данных следующего сегмента гамм H(2).
4. Подсчет контрольной суммы участка данных, соответствующего сегменту данных H(2) и т.д.

2.2 Идея взлома

Шифротексты обеих телеграмм можно получить по формулам режима однократного гаммирования:

$$C_1 = P_1 \oplus K$$
$$C_2 = P_2 \oplus K$$

Открытый текст можно найти, зная шифротекст двух телеграмм, зашифрованных одним ключом. Для это оба равенства складываются по модулю 2. Тогда с учётом свойства операции XOR получаем:

$$C_1 \oplus C_2 = P_1 \oplus K \oplus P_2 \oplus K = P_1 \oplus P_2$$

Предположим, что одна из телеграмм является шаблоном — т.е. имеет текст фиксированный формат, в который вписываются значения полей. Допустим, что злоумышленнику этот формат известен. Тогда он получает достаточно много пар $C_1 \oplus C_2$ (известен вид обеих шифровок). Тогда зная P_1 имеем:

$$C_1 \oplus C_2 \oplus P_1 = P_1 \oplus P_2 \oplus P_1 = P_2$$

Таким образом, злоумышленник получает возможность определить те символы сообщения P_2, которые находятся на позициях известного шаблона сообщения P_1. В соответствии с логикой сообщения P_2, злоумышленник имеет реальный шанс узнать ещё некоторое количество символов сообщения P_2. Затем вновь используется равенство с подстановкой вместо P_1 полученных на предыдущем шаге новых символов сообщения P_2. И так далее. Действуя подобным образом, злоумышленник даже если не прочитает оба сообщения, то значительно уменьшит пространство их поиска.

3 Выполнение работы

Программа запущена как Python-файл в настоящем терминале Ubuntu, без Jupyter. Используется XOR байтов UTF-8 и ключ secrets.token_bytes. Обе строки имеют длину 25 байт. Повторение ключа в двух сообщениях используется только в учебном примере атаки. Шифротексты меняются при каждом новом запуске.

3.1 Реализация XOR и учебной атаки при повторении ключа

```
import secrets
import sys
def xor(a, b):
    if len(a) != len(b):
        raise ValueError('Длины байтовых строк должны совпадать')
    return bytes(x ^ y for x, y in zip(a, b))
p1 = 'КодоваяФраза1'.encode('utf-8')
p2 = 'Безопасность2'.encode('utf-8')
assert len(p1) == len(p2)
key = secrets.token_bytes(len(p1))
c1, c2 = xor(p1, key), xor(p2, key)
print('Архипов Александр Сергеевич | НБИбд-01-24')
print('P1:', p1.decode())
print('P2:', p2.decode())
print('Длина в байтах:', len(p1))
print('C1 (hex):', c1.hex())
print('C2 (hex):', c2.hex())
if len(sys.argv) > 1 and sys.argv[1] == 'attack':
    recovered_key = xor(c1, p1)
    recovered_p2 = xor(xor(c1, c2), p1)
    assert recovered_key == key
    assert recovered_p2 == p2
    print('Восстановленный P2 из C1, C2 и известного P1:', recovered_p2.decode())
    print('Восстановление ключа по известному P1: OK')
else:
    d1, d2 = xor(c1, key), xor(c2, key)
    assert d1 == p1 and d2 == p2
    print('Расшифрованный P1:', d1.decode())
    print('Расшифрованный P2:', d2.decode())
    print('Точное восстановление обоих сообщений: OK')
```

3.2 атака на повторно использованный XOR-ключ

При одном ключе C1 XOR C2 XOR P1 восстанавливает P2; C1 XOR P1 восстанавливает ключ. Обе проверки assert прошли. Шифротексты меняются при новом запуске.

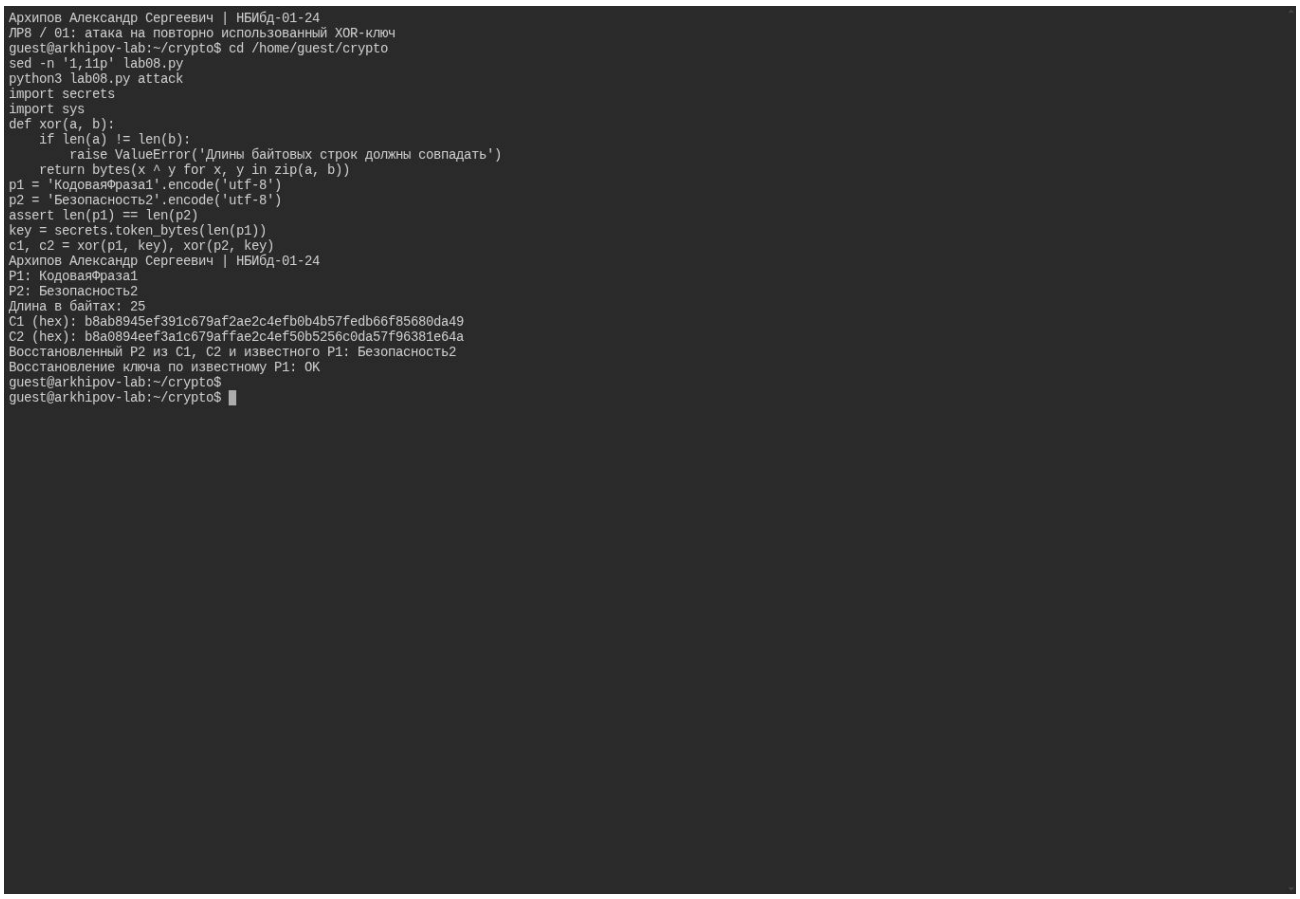


Рис. 1: атака на повторно использованный XOR-ключ

3.3 точное XOR-шифрование и расшифровка UTF-8

XOR применяется к байтам UTF-8; обе фразы имеют длину 25 байт. Обратное преобразование восстанавливает каждую фразу точно, без замены букв. Здесь новый случайный ключ, отличный от снимка атаки.

```
Архипов Александр Сергеевич | НБИбд-01-24
ЛР8 / 02: точное XOR-шифрование и расшифровка UTF-8
guest@arkhipov-lab:~/crypto$ cd /home/guest/crypto
nl -ba lab08.py
python3 lab08.py
     1  import secrets
     2  import sys
     3  def xor(a, b):
     4      if len(a) != len(b):
     5          raise ValueError('Длины байтовых строк должны совпадать')
     6      return bytes(x ^ y for x, y in zip(a, b))
     7  p1 = 'КодоваяФраза1'.encode('utf-8')
     8  p2 = 'Безопасность2'.encode('utf-8')
     9  assert len(p1) == len(p2)
    10  key = secrets.token_bytes(len(p1))
    11  c1, c2 = xor(p1, key), xor(p2, key)
    12  print('Архипов Александр Сергеевич | НБИбд-01-24')
    13  print('P1:', p1.decode())
    14  print('P2:', p2.decode())
    15  print('Длина в байтах:', len(p1))
    16  print('C1 (hex):', c1.hex())
    17  print('C2 (hex):', c2.hex())
    18  if len(sys.argv) > 1 and sys.argv[1] == 'attack':
    19      recovered_key = xor(c1, p1)
    20      recovered_p2 = xor(xor(c1, c2), p1)
    21      assert recovered_key == key
    22      assert recovered_p2 == p2
    23      print('Восстановленный P2 из C1, C2 и известного P1:', recovered_p2.decode())
    24      print('Восстановление ключа по известному P1: OK')
    25  else:
    26      d1, d2 = xor(c1, key), xor(c2, key)
    27      assert d1 == p1 and d2 == p2
    28      print('Расшифрованный P1:', d1.decode())
    29      print('Расшифрованный P2:', d2.decode())
    30      print('Точное восстановление обоих сообщений: OK')
Архипов Александр Сергеевич | НБИбд-01-24
P1: КодоваяФраза1
P2: Безопасность2
Длина в байтах: 25
C1 (hex): 8aac4066e143630c9704c00b550a1d5ba97d3b6aca7fb455dd
C2 (hex): 8aa7406de140630c9709c00b55041d42a8433a5bcb4ab569de
Расшифрованный P1: КодоваяФраза1
Расшифрованный P2: Безопасность2
Точное восстановление обоих сообщений: OK
guest@arkhipov-lab:~/crypto$
guest@arkhipov-lab:~/crypto$
```

Рис. 2: точное XOR-шифрование и расшифровка UTF-8

4 Выводы

В ходе выполнения лабораторной работы было разработано приложение, позволяющее шифровать тексты в режиме однократного гаммирования.

Список литературы

1. Шифрование методом гаммирования
2. Режим гаммирования в блочном алгоритме шифрования